Learning to Speedrun a Long-Horizon Farming Objective from Trajectories

dotrdp
Independent Claude (Anthropic)
AI coding assistant

Abstract. We study how well a policy can learn to **speedrun** a long-horizon collection objective — completing all 24 restoration requests of the Furrow farming simulation as early as possible — primarily from demonstrations. The objective couples seasonal crop windows, weather- and time-gated resources, tool progression and money, and a single missed window costs a season. Using a stateless scripted expert as demonstrator and interactive oracle, we compare behaviour cloning with data scaling, noise-injected demonstrations (DART), filtered and time-conditioned imitation, DAgger, and PPO finetuning with a KL penalty to the imitation prior, all evaluated on 200 paired held-out seeds by the censored completion day. The scripted expert completes the objective on 98.5% of test seeds in 77.9 days on average, and a masked random policy never does. The best learned policy, PPO finetune (from BC+DART), succeeds on 99% of seeds with a mean censored completion day of 76.4. We describe the reward and evaluation choices that make these comparisons meaningful, and a SMAPI bridge and NeMo Gym server that extend the setup to the commercial game the benchmark is modelled on and to LLM agents.
Keywords: imitation learning, DAgger, speedrunning, long-horizon RL, behaviour cloning

1 Introduction

Speedrunning turns a game into an optimisation problem with a single scalar objective: finish as early as possible. Farming/life simulations make this objective unusually long-horizon. Crops planted too late in a season die, some fish bite only in the rain or at night, a wandering merchant's random stock can shortcut an item that would otherwise require waiting for the next season, and an early tool upgrade can unlock deeper ores weeks later. Human speedrunners of the restoration ("community center") objective in the commercial game Stardew Valley plan routes around exactly these constraints.

We ask how much of such a route can be **learned from trajectories**. Following the VPT recipe of imitation followed by RL finetuning [1] and lessons from NetHack, where behaviour cloning from billions of transitions recovers only part of the demonstrator's ability [2], [3], we work in Furrow — an original, deterministic, fast simulator with the same structure (companion paper) — where demonstrations are cheap and the demonstrator can be queried interactively. This lets us separate the effects of data quantity, data diversity, covariate shift and RL finetuning.

2 Problem

An episode starts on day 1 of spring with 500 coins. The agent acts through 192 **macro actions** (temporally extended options [4]: `Plant(crop, field)`, `Fish(spot)`, `Buy(item, q)`, `Deliver`, `Sleep`, ...), each executed by a deterministic controller and masked when its preconditions fail. Observations are 777 item-centric features. The objective is the **completion day** C on which all 24 requests are fulfilled; failures are censored at the 192-day cap. The reward is -1 per in-game day plus potential-based shaping on progress [5], and discounting is per in-game day ($\gamma_{\text{day}} = 0.99$ raised to the number of day boundaries crossed by a transition), so the return-optimal policy is the fastest one and no policy profits from padding a day with extra decisions.

3 Methods

Demonstrator. A stateless priority-rule expert (companion paper) that, for every open request, targets the cheapest candidate slots under a seasonal cost model and executes the first applicable rule. Because it depends only on the state, it provides clean labels on any state, including those visited by a learner.

Data. Three datasets on disjoint training seeds: 1,000 clean expert episodes (1.28 M decisions), 400 episodes with 10% injected random actions but clean labels (DART [6]), and 400 episodes from seven perturbed expert **styles**.

Policy. A 3×512 MLP with layer normalisation over standardised observations (statistics frozen from the data) and masked logits; a separate critic trunk for RL.

Behaviour cloning (BC). Masked cross-entropy on clean labels with label smoothing restricted to valid actions, episode-balanced sampling, 20 k steps of batch 4096. We vary the number of expert episodes (25 ⟶ 1000), add DART data, add style

data, **filter** to the fastest 25% of episodes, and **time-condition** on days-to-go [7] (successful episodes only; at test time we request a target completion day and decrement it as days pass).

DAgger [8]. Starting from BC on 100 episodes, each round rolls out a β-mixture of expert and learner ($\beta_i = 0.5^{i+1}$) in 64 parallel environments for 1,500 decisions, labels all visited states with the expert, aggregates, and retrains; six rounds.

PPO finetuning [9]. From BC+DART: critic warm-up on rollouts of the frozen prior, then PPO with masked log-probabilities, GAE [10] with per-day discounting and bootstrapping from terminal observations on truncation, and a KL penalty to the prior annealed from 0.2 to 0.02 (in the spirit of kickstarting [11]).

Evaluation. 200 held-out test seeds, identical for every method; because exogenous randomness is keyed by seed and day, every method faces the same weather, forage and merchant stock, so per-seed differences to the expert are paired. We report success rate, mean, median and interquartile mean (IQM) of C with bootstrap confidence intervals [12], the speed score $S = \frac{192-\overline{C}}{192-\overline{C}_{\text{expert}}}$ and the paired difference to the expert. Learned policies act greedily.

4 Results

Method	Success	Mean C (95% CI)	IQM C	Median C	Speed	Δ vs expert
Masked random	0.005	192.0 [192.0, 192.0]	192.0	192	0.00	114.1 [110.4, 117.6]
Scripted expert	0.985	77.9 [74.4, 81.6]	69.9	70	1.00	0.0 [0.0, 0.0]
BC (25 ep)	0.355	169.2 [163.8, 174.2]	186.6	192	0.20	91.3 [85.5, 96.5]
BC (100 ep)	0.680	135.8 [129.3, 142.2]	135.6	124	0.49	57.9 [50.3, 65.2]
BC (400 ep)	0.795	114.8 [107.9, 121.7]	102.2	96	0.68	36.9 [30.1, 43.9]
BC (1000 ep)	0.830	103.8 [97.3, 110.3]	86.3	80	0.77	25.9 [19.0, 32.8]
BC + DART	0.930	86.9 [82.3, 91.7]	74.9	72	0.92	9.0 [3.7, 14.5]
BC (mixed styles)	0.950	83.4 [79.0, 88.0]	72.7	72	0.95	5.5 [0.6, 10.5]
Filtered BC (fastest 25%)	0.770	109.2 [102.4, 116.2]	91.2	87	0.73	31.3 [24.3, 38.8]
Time-cond. BC (target 55 d)	0.930	85.6 [80.9, 90.6]	73.0	72	0.93	7.7 [2.3, 13.6]
Time-cond. BC (target 65 d)	0.945	85.3 [80.9, 90.4]	74.2	73	0.93	7.4 [2.6, 12.4]
Time-cond. BC (target 75 d)	0.920	87.1 [82.1, 92.7]	73.7	73	0.92	9.2 [3.9, 14.9]
DAgger	0.915	97.8 [92.2, 103.2]	84.7	82	0.83	19.9 [13.7, 26.1]
PPO finetune (from BC+DART)	0.990	76.4 [73.9, 79.2]	71.9	71	1.01	−1.5 [−5.4, 2.2]

Table 1: Test-seed results (one episode per seed, greedy decoding). C is the censored completion day (lower is better); Δ is the paired per-seed difference to the scripted expert.

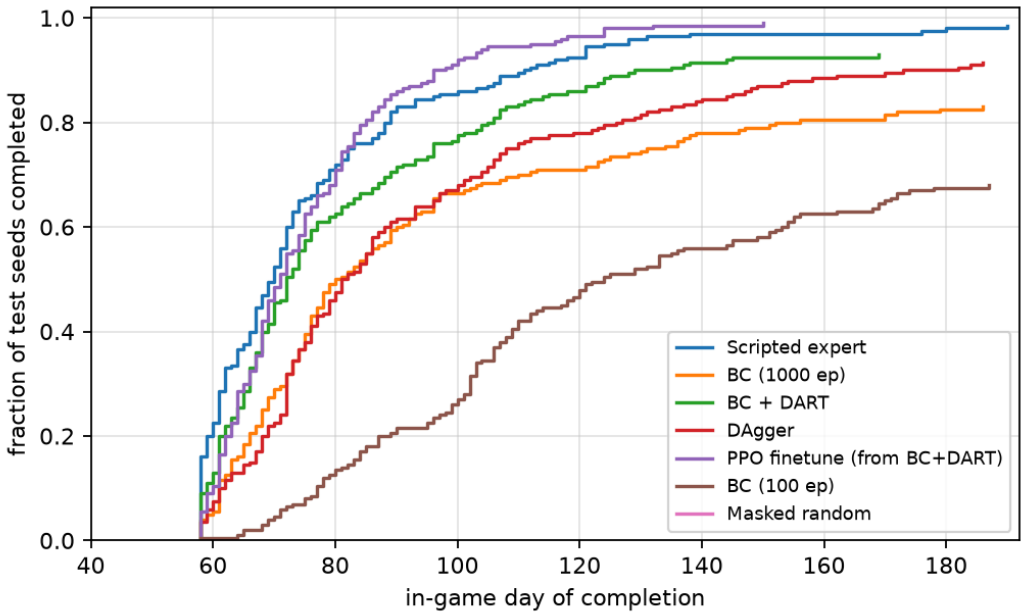


Figure 1: Fraction of test seeds on which the objective is complete by a given day. Curves that saturate below 1 failed on some seeds within 192 days.

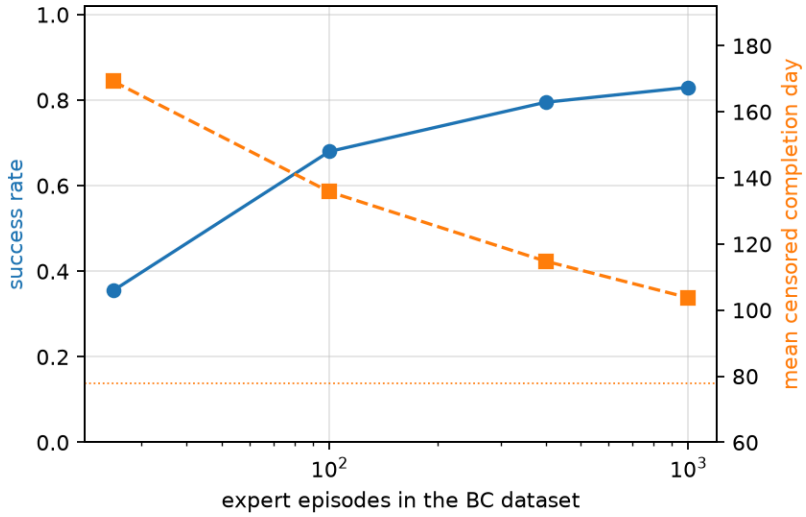


Figure 2: Behaviour cloning versus the number of expert episodes. The dotted line is the expert's mean completion day.

Imitation scales but saturates below the demonstrator. Behaviour cloning improves monotonically with data (Figure 2): success rises from 35.5% with 25 expert episodes to 83% with 1,000, and the mean completion day falls from 169.2 to 103.8. Yet even 1.28 M clean expert decisions leave a paired gap of 25.9 days [19.1, 32.9] to the expert. Most of the gap comes from failed or very late runs — the median completion day is 80.5 versus 70 — which is the compounding-error signature of behaviour cloning on a long horizon: one off-distribution decision early in a season (a crop planted too late, a machine never built) costs a whole season.

Recovery data beats more of the same data. Adding 400 episodes in which 10% of the executed actions were random but the labels stayed clean (DART) lifts success to 93% and shrinks the paired gap to 9 days [3.7, 14.5]; adding style-perturbed expert episodes as well reaches 95% and 5.5 days [0.6, 10.5]. These datasets are larger than the 1,000-episode set, but going from 400 to 1,000 clean episodes only moved success from 79.5% to 83%: states off the expert's path are what is missing.

DAgger is label-efficient. Starting from BC on 100 episodes, 6 rounds of on-policy labelling with the Rust expert (706564 labelled states in total, about half the 1,000-episode set) reach 91.5% test success, versus 68% for its starting point and 83% for BC on 1,000 episodes; validation success rose almost monotonically over the rounds (Figure 3).

Filtering and time conditioning do not help here. Cloning only the fastest 25% of episodes gives 77% success, worse than cloning everything: the filter mostly selects lucky seeds (favourable weather and merchant stock) rather than better decisions, and discards data. Conditioning on days-to-go [7] produces policies indistinguishable from unconditioned BC on the same data (93%–94.5% success, mean C 85.3–87.1), and the requested target barely changes behaviour — consistent with the observation that return conditioning rarely extrapolates beyond the returns present in the data.

RL finetuning closes the gap. PPO finetuning of the BC+DART policy for 60 iterations (128 environments × 128 decisions each, under a minute of wall-clock time on one machine) raises success from 93% to 99% and lowers the mean completion day from 86.9 to 76.4. The finetuned policy is statistically indistinguishable from the scripted expert (paired $\Delta = -1.5$ days, 95% CI $[-5.4, 2.2]$) and faster than it on 37% of the seeds. With a frozen observation normaliser, a critic warmed up on rollouts of the frozen prior, masked log-probabilities and a KL anchor, validation performance did not drop at the start of finetuning (Figure 3, right); we did not ablate these components.

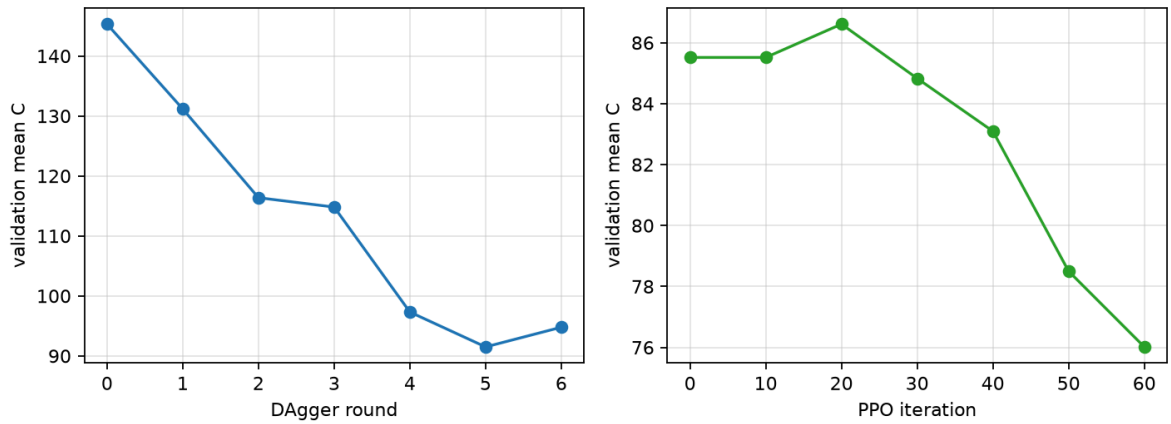


Figure 3: Validation mean completion day over DAgger rounds (left) and PPO finetuning iterations (right).

5 Toward the commercial game and LLM agents

SMAPI bridge. The same agent interface is designed to drive the user's own copy of Stardew Valley through a SMAPI [13] mod — lockstep stepping, structured snapshots including Community Center bundle state, primitive input frames and macro actions executed with the game's own systems — without redistributing any game code or data. It is used to record human trajectories and to evaluate policies; Furrow and the commercial game share the interface shape but not the observation or action semantics, so we make no transfer claim.

LLM agents. An NVIDIA NeMo Gym [14] resources server exposes Furrow as tools (`observe`, `list_actions`, `act`, `status`) over **segments** of the speedrun (a start state taken from an expert trajectory and a seven-day horizon), with a lexicographic reward: any success outranks any failure, and faster successes score higher relative to the expert's time from the same state. Segments keep prompts within a few thousand tokens per decision, which full 1,000-decision episodes would not.

6 Limitations

The demonstrator is a scripted planner, not human; its weaknesses (it sometimes ends days early while waiting for a seasonal window) bound what pure imitation can learn and leave room for RL. Policies are memoryless MLPs; sequence models, entity-centric action heads and offline RL (IQL [15]) are natural next steps. Results are from a single training seed per method; the paired, seeded evaluation keeps comparisons meaningful but does not measure training variance.

Bibliography

[1] B. Baker *et al.*, "Video PreTraining (VPT): Learning to Act by Watching Unlabeled Online Videos," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[2] E. Hambro *et al.*, "Dungeons and Data: A Large-Scale NetHack Dataset," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022.

[3] U. Piterbarg, L. Pinto, and R. Fergus, "NetHack is Hard to Hack," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, pp. 181–211, 1999.

[5] A. Y. Ng, D. Harada, and S. Russell, "Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, 1999.

[6] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "DART: Noise Injection for Robust Imitation Learning," in *Conference on Robot Learning (CoRL)*, 2017.

[7] L. Chen *et al.*, "Decision Transformer: Reinforcement Learning via Sequence Modeling," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[8] S. Ross, G. J. Gordon, and J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[10] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," in *International Conference on Learning Representations (ICLR)*, 2016.

[11] S. Schmitt *et al.*, "Kickstarting Deep Reinforcement Learning," *arXiv preprint arXiv:1803.03835*, 2018.

[12] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[13] Pathoschild, "SMAPI: the Stardew Valley modding API." 2026.

[14] NVIDIA, "NeMo Gym: build RL environments for LLM training." 2026.

[15] I. Kostrikov, A. Nair, and S. Levine, "Offline Reinforcement Learning with Implicit Q-Learning," in *International Conference on Learning Representations (ICLR)*, 2022.